ε-Sufficient Context for Bounded Readers: Definitions and a Non-Monotonicity Result for Code Editing*

Jude Castiel Johansen
Shockware Limited, independent researcher
castiel@shockware.co.uk
ORCID 0009-0002-5644-3073

October 2026

Abstract

Context retrieval for a language model that edits code is evaluated almost entirely by end-task pass rate. There is no agreed definition of what context is *sufficient*, and the natural baseline, the principle of total evidence, assumes a reader for whom cost-free information never hurts. Language models are not such readers: their accuracy falls as input grows. This note fixes definitions that take that into account. A change request is a contract (Δ, D, I, W) with an explicit frame condition and a write-set, checked through an oracle V_C whose unsoundness is named and kept separate. A context K is *ε-sufficient* for an editor A when A's success on K is within ε of its success on the best context in a stated candidate family, not on the full repository. Within a stylised coverage-times-degradation model we prove, and mechanise in Lean 4, that ε-sufficiency is not closed under superset, with a separation that grows with the amount of irrelevant material, while it is closed under superset for a lossless reader. We state the forward slice as a requirement on the oracle, give three design properties that would make the context a graph query, and report a mutation-based measurement on a real Go repository of how often an interface-preserving change is caught only by tests outside the changed package, together with the token cost of showing those packages to an editor. A planned benchmark, SuffBench, is described. No language-model experiments are reported.

1 Problem

A code change is produced by an editor from a request and some portion of the repository. For a Bayes-rational reader, the portion can be large without penalty: by Good's principle of total evidence, cost-free information never lowers expected decision quality [14, 5]. Program slicing [39, 15] inherits the spirit of that assumption and asks only whether a portion contains enough; where slices fail to be closed under superset, as observation-based slices do [3], the cause is well-formedness, not the reader's capacity.

A language model is a different reader. It reads under a window, and its accuracy degrades with input length and with irrelevant content [27, 24, 16, 36], including when retrieval is perfect and the padding is neutral [11]. Retrieval-augmented generation shows the same shape: more retrieved passages can lower accuracy [20, 42], and the direction of the effect is not even uniform [8]. Measurements on coding agents in 2026 point the same way: what decides success is whether the needed facts are present, not how much surrounds them [35, 28], and long contexts hurt whether or not their content is relevant [41]. For such a reader the question is not whether a portion contains enough but whether it contains enough and not too much, and the two cannot be separated.

The term *sufficient context* has been used for retrieval-augmented generation by Joren et al. [21], where it denotes an information-content property of the context alone. The notion here is different in two respects, and the prefix ε marks the difference: it is relative to the reader, and it is relative to the best achievable context under an external correctness oracle rather than to the answer. Section 10 places both against the statistical and explainability literature on approximate and minimal sufficiency.

The gap this note addresses, stated narrowly: no prior definition of minimal context for code edits combines an external correctness oracle with a frame condition, comparison to the best achievable context, and a reader whose success is non-monotone in context size.

*Version 1.1. Relative to version 1.0 (DOI 10.5281/zenodo.23282196, 10 October 2026) this version adds, in Sections 6, 8, 9 and 10, citations to and positioning against work on context for coding agents that appeared between April and August 2026, and points to the public SuffBench specification. Definitions, proofs, and measurements are unchanged.

2 Setting

A codebase is a graph $G = (U, E)$. U is a finite set of *units*, such as files, functions, or packages. E is a set of typed edges, such as calls, imports, data flow, and tests-cover. A program P assigns a body to each unit. Its observable behaviour is a map $[\![P]\!] : X \to Y$ from inputs to outputs or traces. Φ is a set of predicates over programs, including tests, type judgements, and stated invariants. A check $\varphi \in \Phi$ is *bound to* a unit u if u is among the units whose code the check exercises; for tests this is the package or file under test.

A *context* is a subset $K \subseteq U$. A reader sees a sequence, not a set, so we fix a canonical serialisation $\sigma(K)$ and write $\mathrm{load}(K)$ for its length in tokens. Load is monotone under inclusion. A bounded reader has a window L; any model of its success should satisfy $g(\ell) = 0$ for $\ell > L$, and we will not need more than that.

3 The contract and its oracle

Definition 1 (Contract)**.** A change request is a tuple $C = (\Delta, D, I, W)$ with $\Delta \subseteq \Phi$ the requested behaviour, $D \subseteq X$ the scope of inputs on which behaviour may differ, $I \subseteq \Phi$ the invariants, and $W \subseteq U$ the *write-set*, the units the edit is permitted to change. A program P' *satisfies* C relative to the current program P, written $P' \models C$, iff

(a)	$\forall \varphi \in \Delta.\ \varphi(P')$	requested behaviour
(b)	$\forall x \notin D.\ [\![P']\!](x) = [\![P]\!](x)$	frame: nothing else changes
(c)	$\forall \varphi \in I.\ \varphi(P) \Rightarrow \varphi(P')$	invariants preserved
(d)	$\forall u \notin W.\ \mathrm{body}_{P'}(u) = \mathrm{body}_{P}(u)$	edit confined to the write-set.

C is *well-defined* iff some P' satisfies it, and *precise* iff any two satisfying programs agree on D. Imprecision is the formal name for an ambiguous request. Condition (b) is the frame problem of procedure specifications [6]; it is analogous to, not an instance of, the frame rule of separation logic [29], whose footprint is heap rather than input domain. Condition (d) is what makes the write-set meaningful: a localisation step proposes W before context is selected, and everything downstream, in particular the forward slice of Section 6, is a function of it. When the request permits interfaces to change, the subset $W_I \subseteq W$ of units whose declared interfaces may change is recorded as well.

Satisfaction cannot be checked directly. What is run is an *oracle* $V_C \subseteq \Phi$: the tests for Δ, the checks for I, and whatever regression tests approximate (b). We write $V_C(P') = \mathsf{pass}$ when every check holds. The oracle is *sound* for C when $V_C(P') = \mathsf{pass}$ implies $P' \models C$, and *complete* when $P' \models C$ implies $V_C(P') = \mathsf{pass}$. Real oracles are neither: tests over-specify error text and reject correct alternative patches, so completeness fails, and tests cannot cover all of $X \setminus D$, so soundness fails. The *oracle residual* is the unsoundness set $\{P' : V_C(P') = \mathsf{pass}\} \setminus \{P' : P' \models C\}$. Everything measured in this note is measured against V_C; the residual is real, unmeasured, and must be reported as such.

4 Sufficiency for a bounded, stochastic editor

An *editor* A maps a contract and a serialised context to a distribution over programs $A(C, K)(P)$. Fix a finite *candidate family* $\mathcal{K} \subseteq 2^U$ of contexts under consideration. Write

$$p_A(K, C) = \Pr\big[\, V_C\big(A(C, K)(P)\big) = \mathsf{pass} \,\big], \qquad p^*_{A,\mathcal{K}}(C) = \max_{K \in \mathcal{K}} p_A(K, C).$$

Definition 2 (ε-sufficient context)**.** $K \in \mathcal{K}$ is (ε, A)-*sufficient* for C relative to $\mathcal{K}$ iff $p_A(K, C) \geq p^*_{A,\mathcal{K}}(C) - \varepsilon$. Given a cost on contexts, by default $\mathrm{cost}(K) = \mathrm{load}(K)$, K is *cost-minimum* iff it is sufficient and no sufficient $K' \in \mathcal{K}$ has strictly lower cost.

Three points distinguish this from the slicing notion. Success is a probability, not a verdict. The comparison is to the best context in the family, not to the full repository, because the full repository may itself be a poor choice. And nothing in the definition makes sufficiency closed under superset; because it is not, inclusion-minimal sufficient sets are badly behaved, which is why the definition asks for cost-minimum instead.

On ε. With $\mathcal{K} = 2^U$ the maximum is over a finite set and $\varepsilon = 0$ is well-defined, but it cannot be estimated: $\ln|\mathcal{K}| = |U| \ln 2$. In practice $\mathcal{K}$ is small, polynomial in $|U|$, for instance the slices of Section 6 at a few radii. The tolerance is then needed because both $p_A(K, C)$ and p^* are estimated, each from n samples: the first carries a Hoeffding term of order $\sqrt{\ln(1/\delta)/(2n)}$, the second a union-bound term of order $\sqrt{\ln(|\mathcal{K}|/\delta)/(2n)}$ together with the usual upward bias of a selected maximum. The oracle residual is not part of ε; it is a separate gap between V_C and $\models C$.

Assumption 1 (Coverage and degradation)**.** The reader has a *degradation curve* $g_A : \mathbb{N} \to [0, 1]$, non-increasing in load and determined independently of any particular repository (Hypothesis 1). Given g_A, the *coverage* of a context is defined as $r(K, C) = p_A(K, C)/g_A(\mathrm{load}(K))$, and the assumption is that $r \leq 1$. A set S is *irrelevant* to K when $r(K \cup S, C) = r(K, C)$.

Defining coverage as the ratio makes the factorisation $p = r \cdot g$ identified rather than free: whether a set is irrelevant is checkable once g_A is pinned down, and only then. This is a stylised model and it is known to be false in its sharpest forms. Coverage is a property of the editor's needs, not of the verdict: SWE-bench [19] withholds the fail-to-pass tests, which certainly determine the verdict, and editors still succeed. So coverage must not be read as containment of a verdict-determining slice, and the binary form $r = [O \subseteq K]$ for a fixed O is only the simplest instance. Nothing in Section 5 depends on the shape of r beyond the existence of irrelevant units.

Hypothesis 1 (Portability of the degradation curve)**.** g_A can be estimated once per model from planted-fact experiments and reused across codebases.

Long-context benchmarks show degradation varying with the task [16, 24]; if the hypothesis fails, g_A must be estimated per task family, which raises the cost of applying the definitions but does not change them.

5 Non-monotonicity

We now make Assumption 1 concrete and prove that sufficiency is not closed under superset, in contrast with the lossless case. Both statements, the fact that the optimum is attained, and a concrete two-unit witness are mechanised in Lean 4 with Mathlib, with the optimum defined as a maximum over all contexts, load taken to be $|K|$, and values in $\mathbb{Q}$.[1]

Fix a finite set of units U, a coverage function $r : 2^U \to [0, 1]$ of any shape, and a degradation curve $g : \mathbb{N} \to [0, 1]$. Let $p(K) = r(K)\, g(|K|)$, let $p^* = \max_{K \subseteq U} p(K)$, which is attained, and say K is ε-sufficient iff $p(K) \geq p^* - \varepsilon$.

Proposition 1 (Sufficiency is not closed under superset)**.** *Let g be strictly decreasing. Let K attain p^*, let $r(K) > 0$, and let $S \neq \emptyset$ be disjoint from K and irrelevant to it. Then $p(K \cup S) < p(K)$, the drop equals $r(K)\,\big(g(|K|) - g(|K| + |S|)\big)$, and for every ε with $0 \leq \varepsilon$ below the drop, the context K is ε-sufficient and the context $K \cup S$ is not.*

Proof. Irrelevance gives $r(K \cup S) = r(K)$, and disjointness gives $|K \cup S| = |K| + |S| > |K|$, so $p(K) - p(K \cup S) = r(K)\,(g(|K|) - g(|K| + |S|)) > 0$ because g is strictly decreasing and $r(K) > 0$. Since $p(K) = p^*$, K is ε-sufficient for every $\varepsilon \geq 0$. For $K \cup S$, $p(K \cup S) < p(K) - \varepsilon = p^* - \varepsilon$ by the bound on ε. □

The admissible interval for ε is non-empty and its width grows with $|S|$: the more irrelevant material is added, the larger the tolerance at which the superset still fails. The one-unit case is the weakest instance, not the result. For the binary instance $r = [O \subseteq K]$ with g non-increasing, the optimum is $g(|O|)$, attained at $K = O$. Concretely, with $g(\ell) = 1/(\ell + 1)$ and $|O| = 1$, adding three irrelevant units drops success from $1/2$ to $1/5$, so every $\varepsilon < 3/10$ separates O from $O \cup S$; the two-unit case is the mechanised witness.

Lemma 1 (The lossless reader is closed under superset)**.** *Let g be constant and let r be monotone under inclusion. If K is ε-sufficient and $K \subseteq K'$, then K' is ε-sufficient, for every ε.*

Proof. With $g \equiv c \geq 0$, $p(K') = r(K')\, c \geq r(K)\, c = p(K) \geq p^* - \varepsilon$. □

[1] File `sufficiency/Sufficiency/NonMonotonic.lean`, commit `b372f3222686`, in the repository below; theorem `prop1` is the Proposition with exactly the hypotheses stated here. The development contains no `sorry`; CI audits that every theorem depends only on `propext`, `Classical.choice` and `Quot.sound`.
`https://github.com/Shockware-Limited/context-sufficiency-note`

Monotone coverage is an assumption of the Lemma, and Good's principle is the reason to make it: for a reader who pays nothing to read, more information never lowers coverage. Under that assumption a constant g gives superset closure, and minimality is a matter of removing what is unnecessary. A decreasing g makes adding the unnecessary a failure mode, and minimality becomes a cost-minimisation whose objective is non-monotone in the context.

6 The forward slice as a requirement on the oracle

Let W be the write-set. The *backward slice* $B(W)$ is the set of units W depends on, transitively, through the edges of G. The *forward slice* $F(W)$ is the set of units that depend on W, transitively, including through test-only dependencies. Retrieval for code editors has concentrated on the backward slice and on the edit site itself [37, 45, 40, 2].

Hypothesis 2 (Compositionality and soundness of the dependency relation)**.** The dependency relation is well-founded, or the semantics is a least fixpoint over it; the set of behaviours of a unit is determined by its own body and the behaviours of the units it depends on; and the relation in E over-approximates every channel through which one unit's behaviour can affect another's.

Under Hypothesis 2 and condition (d), a unit outside $W \cup F(W)$ has the same body and the same dependency behaviours before and after the edit, so its behaviour is unchanged. Every violation of condition (b) therefore shows up in the behaviour of a unit in $W \cup F(W)$, and the checks that can observe it are the checks bound to those units. Regression test selection [34] is safe under an analogous assumption that the dependence representation captures every code path, together with Rothermel and Harrold's controlled regression testing assumption that everything other than the code is held fixed between runs; change impact analysis [33] computes the same set, and supplying such an impact map to a coding agent has been reported to reduce the regressions it introduces [1].

The hypothesis fails in practice through channels the dependency graph does not record: shared mutable or global state, environment and configuration, initialisation order, registration through side-effecting imports or dependency injection, build tags and generated code, reflection and serialisation, timing, and consumers outside the repository altogether. Which of these escape a given graph depends on its granularity; Section 8 discusses the case of Go's package graph.

Requirement 1 (Forward slice)*.* An oracle V_C that omits the checks bound to $F(W) \setminus W$ cannot observe a violation of condition (b) whose only witnesses lie there. Soundness of V_C for the frame therefore requires those checks; it does not follow from them.

This is a statement about the oracle, and it follows from the hypothesis. A further claim, that an editor which can *see* the dependents it may break will break them less often, is an empirical claim about editors; nothing in this note tests it, and the token costs reported below are the cost of that untested step.

7 Context-closed languages

The quantities above are undecidable in general: whether a unit can affect the verdict on an arbitrary program is a semantic property, and minimal slices are uncomputable [39]. They become graph queries when the language restricts programs. We state three design properties for such a restriction; they are desiderata for a core calculus that does not yet exist, and no language is named. Extend U with an *interface node* $\mathrm{spec}(v)$ for each unit v, carrying its declared contract and effects, and write $\mathrm{deps}(u)$ for the units u depends on and Φ_u for the set of behaviours permitted to u.

Property 1 (P1, interface determinacy)**.** If every body refines its specification, then Φ_u is determined by $\mathrm{body}(u)$ and the family $(v \mapsto \mathrm{spec}(v))_{v \in \mathrm{deps}(u)}$. This is modular reasoning in the sense of Parnas [30] and behavioural subtyping [26], taken as a language guarantee rather than a discipline.

Property 2 (P2, spec-bounded reliance)**.** If every check bound to a unit other than u depends on u only through $\mathrm{spec}(u)$, and an edit leaves $\mathrm{spec}(u)$ unchanged with the new body refining it, then the verdict of every such check is unchanged. The antecedent fails whenever a dependent relies on behaviour the specification leaves open, such as iteration order or error text; P2 is a property of the language's ability to forbid that reliance, and it says nothing about condition (b) within the latitude the specification leaves.

Property 3 (P3, computable context)**.** For a contract with write-set W and interface write-set W_I, let K^* be the least set containing W, the interface nodes of $\mathrm{deps}(W)$, and, for every unit whose interface

node is in K^* and lies in W_I or depends on W_I, the units depending on it together with their interface nodes, closed under this rule, plus the checks bound to any unit in K^*. Given G and syntactic equality of interfaces, K^* is computable before the edit and in time linear in $|G|$; constructing G is not included in that bound.

A language is *context-closed* iff P1, P2 and P3 hold of all its programs. In such a language the backward slice stops at one hop of interfaces, the forward slice is empty when $W_I = \emptyset$, and the undecidable question is replaced by a fixpoint over interface changes. Note that K^* contains interface nodes where an observation-based oracle slice would contain bodies; the two agree only if the checks on dependents rely on the specification alone, which is P2. The ingredients exist individually, as declared contracts, declared effects, content-addressed definitions and acyclic modules; the property as a language guarantee is what is proposed.

8 Measurement: how often does the forward slice matter?

The one claim in this note that can be tested without a language model is the one behind Requirement 1: that interface-preserving changes are caught by tests outside the changed unit. In a language that is not context-closed this is also a direct measurement of how often P2's antecedent fails, that is, how often dependents rely on behaviour no interface pins down. We measured it on the GitHub CLI repository (`cli/cli`, commit `ec5b512045db`), a single Go module of 310 packages, with go1.27.2 windows/amd64 on Windows, taking the package as the unit, since it is Go's unit of compilation and of testing and its import graph is acyclic. Of its packages, 237 have tests that pass at baseline; 13 fail at baseline and are excluded throughout, without attributing a cause. The full suite runs in about 163 seconds.

What the package graph can and cannot see. At package granularity in Go, $F(W)$ is exactly the set of packages whose test binaries link W. Every in-process channel listed in Section 6, including global state, initialisation order, and reflection, lives inside a linked binary and is therefore inside $F(W)$ by construction. A failure outside $W \cup F(W)$ can only arise from a cross-process channel, such as a test executing a repository-built binary, reading source files as data, or sharing the file system or environment with concurrently running test packages, or from flakiness. This design therefore cannot measure in-process violations of Hypothesis 2, and a small *elsewhere* count must not be read as support for it.

Design. We sampled packages uniformly among those with non-test Go files and at least one reachable passing test package, then a non-test file within the package, then a mutation site uniformly within the file, and injected exactly one mutant. The operators are: negation of a relational operator ($<$ to $\geq$ and so on, and $=$ to $\neq$); $+$ to $-$ and back; $\wedge$ to $\vee$ and back; negation of an `if` condition; incrementing an integer literal; appending a character to a string literal. All are body-only and preserve every signature and exported name. The budget of 120 sampled mutants includes those that do not compile, which are recorded and excluded. For each compiling mutant we ran the full suite once, with the Go toolchain's default vet checks enabled, recorded the set of failing test packages, and reverted. After the campaign, every package that failed on any mutant was rerun three times on the clean tree; packages failing on the clean tree are flaky and are removed from every fail set before classification (0 such packages). Classes are exclusive: *caught by* W if W's own tests fail; *caught only in* $F(W)$ if W's tests pass and some test package in $F(W)$ fails; *elsewhere only* if the only failures lie outside $W \cup F(W)$; *undetected*; *timeout*. Context sizes are computed exactly over all 309 candidate packages, in tokens of a public tokenizer (cl100k_base), with source and tests counted separately. Driver, mutator, raw results, flake-control output and analysis scripts are in the repository under `experiments/`.

Results. Of 120 sampled mutants, 118 compiled and 72 of those were caught by the full suite (61% [52, 69]). Table 1 gives the exclusive breakdown with Wilson 95% intervals. Among caught mutants, 92% [83, 96] were caught by the mutated package's own tests, 8% [4, 17] were caught only by tests in the forward slice, and 0% [0, 5] failed only elsewhere. The forward-slice-only rate is the number that bears on Requirement 1, and it must be stratified: among caught mutants in packages that have tests of their own (69), the rate is 4% [1, 12]; among those in packages without tests (3), it is 100% [44, 100], where it is automatic. 10 mutants were caught both by W and by $F(W)$.

On cost, over all 309 candidate packages the median package is 1,390 tokens of source; its backward slice is 112k tokens, its forward slice 22k tokens of source plus 19k of tests (90th percentile of the forward slice: 133k), against 915k tokens of source and 1392k of tests for the module. The forward slice is empty

Sampled mutants (incl. non-compiling)	120
compiled	118
caught by the full suite, after flake control	72 (61% [52, 69])
Caught by W's own tests	92% [83, 96]
Caught only by tests in $F(W)$	8% [4, 17]
among packages with their own tests ($n = 69$)	4% [1, 12]
among packages without tests ($n = 3$)	100% [44, 100]
Failed only outside $W \cup F(W)$	0% [0, 5]
Median tokens: W / $B(W)$ / $F(W)$ / tests of $F(W)$	1,390 / 112k / 22k / 19k
Median transitive-slice context as % of module	8%
Fits 128k / 200k window	6% / 63%

Table 1: Mutation measurement on `cli/cli` at commit `ec5b512045db`. Percentages of caught mutants are exclusive classes with Wilson 95% intervals in brackets. Token statistics are exact over all candidate packages. One repository, one platform.

for 1% of packages and has a median of 6 packages. The transitive-slice context $W \cup B(W) \cup F(W)$ with its tests is a median 8% of the module (90th percentile 29%); 6% of packages have one that fits a 128k-token window and 63% a 200k window.

What this does and does not show. The forward-slice-only rate among packages with their own tests is the share of detected interface-preserving faults that an oracle restricted to W's own tests misses and the tests of $F(W)$ catch. In this repository that share is small: about one caught fault in twenty-three, with an interval that excludes zero but not much more. The forward slice's oracle value here is real but marginal, and P2's antecedent, that dependents rely only on what the interface pins down, holds most of the time. Where it fails is informative: every forward-slice-only detection was in a shared helper package (SSH and browser helpers, documentation generation, command-group `shared` packages) whose own tests are thin and whose behaviour several commands depend on. That is the profile of a unit for which the forward slice is worth paying for. The rate is an oracle-side number, whose cost is test execution time, and it is at the same time the rate at which P2's antecedent fails. The token columns are the cost of the further, untested, step of showing $F(W)$ to an editor. No failure was observed outside $W \cup F(W)$ and no flaky package was found among the 72 detections; the former counts cross-process channels only, and says nothing about in-process violations of Hypothesis 2, which this design cannot see. None of this is an editor pass rate. The numbers relate to what regression test selection studies measure [23, 13] and to the regression reduction reported when agents are given graph-based impact maps [1]; the new part is the reading as context cost.

Threats. Mutants are not real regressions, and the operator set is small and unbalanced: sites are drawn uniformly within a file, so string-literal and `err != nil` sites dominate, and the string + to − swap never compiles, inflating the non-compiling count. One repository, one commit, one platform, with 13 packages excluded at baseline. Flake control uses three clean-tree reruns and cannot catch rarer flakes; test packages run concurrently and may interfere through the file system or environment; the test environment inherited the author's home directory. Vet findings triggered by a mutant count as detections in W. Timeouts count as undetected. Package granularity inflates both slices relative to a symbol-level analysis, and the transitive forward slice is an upper bound on what the oracle needs under Hypothesis 2, not on what an editor needs. Sampling by package weights small packages heavily. Token counts use one public tokenizer as a proxy. The mutator binary was built from `experiments/mutator` at the recorded commit.

9 Proposed benchmark

We reserve the name *SuffBench* for a benchmark whose instances are tuples (repository snapshot, contract C derived from a fail-to-pass commit including its write-set, gold patch P'_{gold}, oracle slice $O(C, P'_{\text{gold}})$). The oracle slice is relative to the gold patch, since different correct patches have different slices, and is computed by observation: stub each unit outside the write-set, rebuild, rerun V_C, and retain the unit iff the verdict changes, using delta debugging [44] to reduce the number of builds; the result is 1-minimal and under redundancy order-dependent, so the order is recorded. Instances must postdate the training cutoff of any evaluated model. Existing benchmarks of context retrieval for coding agents label relevance from

agent workflow signals or from the files a gold patch touches [31, 25]; SuffBench's slices are verdict-based, and its frame sets are the pass-to-pass tests of SWE-bench and its multilingual successor [19, 43], kept separate from the fail-to-pass set. The dataset labels *verification* relevance; experiments are to measure *generation* success, and Assumption 1 does not identify the two. Claims are to be stated as "within ε of the best observed pass rate in family $\mathcal{K}$ under oracle V_C" with the residual reported as unmeasured. The instance format and datasheet are specified at `https://github.com/Shockware-Limited/suffbench`; no instances are released with this note.

10 Related work

Sufficiency with a lossless reader. Slicing [39, 15] and its observation-based, language-independent form [3] compute sets of statements sufficient to reproduce a criterion. In statistics, sufficiency and its approximate form are Le Cam's [22]; the monotonicity of cost-free information for a rational agent is Good's [14] and Blackwell's [5]. Lemma 1 is that monotonicity in the present model; Proposition 1 is its failure for a bounded reader.

Sufficiency relative to a model. Explainability measures sufficiency and comprehensiveness of rationales relative to a classifier's own output [10], and formal explanation methods compute minimal sufficient subsets of features for a decision [17, 9]. The present notion differs in comparing to the best context under an external oracle rather than to the model's full-input behaviour, and in the frame condition. Joren et al. [21] define sufficient context for retrieval-augmented generation as a property of the context's information content; ours is reader-relative and optimum-relative.

Non-monotone retrieval. That more context can hurt a language model is established for position [27], length even under perfect retrieval [24, 16, 11, 32], irrelevant content [36, 42], and passage count [20]; noise can even help [8], which is why g being non-increasing is an assumption here and not a fact.

Concurrent work on context for coding agents. Between April and August 2026 several measurements appeared that bear directly on this note and postdate its drafting. Sam-Bodden [35] holds localisation fixed with an oracle on SWE-bench Verified and varies the representation of surrounding code, finding that signatures and skeletons serve about as well as source and that the surrounding context hardly matters; this anticipates, for Python, what Property P1 asks of a language. Mohammadi et al. [28] supply and withhold facts across seven models and find that availability decides the outcome and distance does not, which is coverage dominating load in the terms of Assumption 1. Xue [41] reports a large drop from a short to a long context on an auditing task whether the padding is relevant or not, on two tasks. Alonso et al. [1] reduce agent regressions with a graph-based impact map, the agent-side counterpart of Requirement 1. Jia, Barr and Mechtaev [18] compress issue-resolution context by searching for a minimal sufficient subset on the editor side; pruning inside the agent [38] and compression of repository context [12, 46] improve or preserve task quality at lower cost, while an attribution of where compression saves money in multi-turn agents is given in [7]. Benchmarks of repository context retrieval for agents [31, 25] label relevance from workflow signals or gold-patch files rather than from test verdicts. Run-to-run randomness in agent evaluation, which bounds what any comparison of contexts can resolve, is quantified in [4]. None of these defines sufficiency; they measure it, and their findings are consistent with the model here: coverage first, degradation second, and the frame checked only by the dependents' tests.

Context for code editors and change impact. Context is selected by learned prompt proposals [37], iterative retrieval [45], simple localisation [40], and dependency-graph planning [2]; none states a sufficiency criterion, and all compare against task pass rate on benchmarks such as SWE-bench [19]. Safe regression test selection [34, 13, 23] and change impact analysis [33] compute which checks a change can affect, which is the oracle half of Requirement 1. Modular reasoning through interfaces is Parnas [30] and Liskov and Wing [26]; P1 to P3 ask for it as a language guarantee.

Acknowledgements and AI-use disclosure

The author used Anthropic's Claude Fable 5.1, accessed in October 2026 and referred to by the author as *Lemma*, to draft and revise the text of all sections, to write the Lean 4 development in `sufficiency/`, to write the mutation tooling and analysis scripts in `experiments/` and run the measurement, and to cross-check bibliographic records against Crossref, PMLR and arXiv. A separate model, Claude Opus, was used to produce two rounds of critical review of drafts. The author conceived the problem, its framing and the benchmark, directed the work, reviewed all AI-generated text, proofs, code and results, decided which

review comments to adopt, and re-ran the Lean build and the analysis before publication. Generative AI tools are not authors of this work; the author takes full responsibility for its content.

References

[1] Pepe Alonso, Sergio Yovine, and Victor A. Braberman. TDAD: Test-driven agentic development: Reducing code regressions in AI coding agents via graph-based impact analysis. arXiv:2603.17973, 2026.

[2] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D. C., Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. CodePlan: Repository-level coding using LLMs and planning. *Proceedings of the ACM on Software Engineering*, 1(FSE):675–698, 2024.

[3] David Binkley, Nicolas Gold, Mark Harman, Syed Islam, Jens Krinke, and Shin Yoo. ORBS: Language-independent program slicing. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2014)*, pages 109–120, 2014.

[4] Bjarni Haukur Bjarnason, André Silva, and Martin Monperrus. On randomness in agentic evals. arXiv:2602.07150, 2026.

[5] David Blackwell. Equivalent comparisons of experiments. *The Annals of Mathematical Statistics*, 24(2):265–272, 1953.

[6] Alexander Borgida, John Mylopoulos, and Raymond Reiter. On the frame problem in procedure specifications. *IEEE Transactions on Software Engineering*, 21(10):785–798, 1995.

[7] Luzhuo Chen and Jiayu Shi. An empirical cost attribution of context-compression gateways in multi-turn coding agents. arXiv:2609.22114, 2026.

[8] Florin Cuconasu, Giovanni Trappolini, Federico Siciliano, Simone Filice, et al. The power of noise: Redefining retrieval for RAG systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2024)*, pages 719–729, 2024.

[9] Adnan Darwiche and Auguste Hirth. On the reasons behind decisions. arXiv:2002.09284, 2020.

[10] Jay DeYoung, Sarthak Jain, Nazneen Fatema Rajani, Eric Lehman, et al. ERASER: A benchmark to evaluate rationalized NLP models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, pages 4443–4458, 2020.

[11] Yufeng Du, Minyang Tian, Srikanth Ronanki, Subendhu Rongali, et al. Context length alone hurts LLM performance despite perfect retrieval. arXiv:2510.05381, 2025.

[12] Jia Feng, Zhanyue Qin, Cuiyun Gao, Ruiqi Wang, et al. On the effectiveness of context compression for repository-level tasks: An empirical investigation. arXiv:2604.13725, 2026.

[13] Milos Gligoric, Lamyaa Eloussi, and Darko Marinov. Practical regression test selection with dynamic file dependencies. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA 2015)*, pages 211–222, 2015.

[14] I. J. Good. On the principle of total evidence. *The British Journal for the Philosophy of Science*, 17(4):319–321, 1967.

[15] Susan Horwitz, Thomas Reps, and David Binkley. Interprocedural slicing using dependence graphs. *ACM Transactions on Programming Languages and Systems*, 12(1):26–60, 1990.

[16] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, et al. RULER: What's the real context size of your long-context language models? arXiv:2404.06654, 2024.

[17] Alexey Ignatiev, Nina Narodytska, and Joao Marques-Silva. Abduction-based explanations for machine learning models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 1511–1519, 2019.

[18] Haoxiang Jia, Earl T. Barr, and Sergey Mechtaev. Compressing code context for LLM-based issue resolution. arXiv:2603.28119, 2026.

[19] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, et al. SWE-bench: Can language models resolve real-world GitHub issues? arXiv:2310.06770, 2023.

[20] Bowen Jin, Jinsung Yoon, Jiawei Han, et al. Long-context LLMs meet RAG: Overcoming challenges for long inputs in RAG. arXiv:2410.05983, 2024.

[21] Hailey Joren, Jianyi Zhang, Chun-Sung Ferng, et al. Sufficient context: A new lens on retrieval augmented generation systems. arXiv:2411.06037, 2024.

[22] Lucien Le Cam. Sufficiency and approximate sufficiency. *The Annals of Mathematical Statistics*, 35(4):1419–1455, 1964.

[23] Owolabi Legunsen, Farah Hariri, August Shi, Yafeng Lu, et al. An extensive study of static regression test selection in modern software evolution. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2016)*, pages 583–594, 2016.

[24] Mosh Levy, Alon Jacoby, and Yoav Goldberg. Same task, more tokens: The impact of input length on the reasoning performance of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*, pages 15339–15353, 2024.

[25] Han Li, Letian Zhu, Bohan Zhang, Rili Feng, et al. ContextBench: A benchmark for context retrieval in coding agents. arXiv:2602.05892, 2026.

[26] Barbara H. Liskov and Jeannette M. Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6):1811–1841, 1994.

[27] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[28] Bardia Mohammadi, Lars Klein, Aman Chadha, Akhil Arora, et al. The working set of a coding agent: Coherence debt in repository-scale tasks. arXiv:2608.16630, 2026.

[29] Peter W. O'Hearn, John C. Reynolds, and Hongseok Yang. Local reasoning about programs that alter data structures. In *Computer Science Logic (CSL 2001)*, volume 2142 of *Lecture Notes in Computer Science*, pages 1–19, 2001.

[30] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

[31] Bowen Qin and Yi Xie. Agent retrieval bench: Evaluating repository context retrieval for coding agents. arXiv:2607.24882, 2026.

[32] Stefano Rando, Luca Romani, Alessio Sampieri, Luca Franco, et al. LongCodeBench: Evaluating coding LLMs at 1M context windows. arXiv:2505.07897, 2025.

[33] Xiaoxia Ren, Fenil Shah, Frank Tip, Barbara G. Ryder, and Ophelia Chesley. Chianti: A tool for change impact analysis of Java programs. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2004)*, pages 432–448, 2004.

[34] Gregg Rothermel and Mary Jean Harrold. A safe, efficient regression test selection technique. *ACM Transactions on Software Engineering and Methodology*, 6(2):173–210, 1997.

[35] Brian Sam-Bodden. What context does a coding agent actually need to act? arXiv:2607.09691, 2026.

[36] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, volume 202 of *Proceedings of Machine Learning Research*, pages 31210–31227, 2023.

[37] Disha Shrivastava, Hugo Larochelle, and Daniel Tarlow. Repository-level prompt generation for large language models of code. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, volume 202 of *Proceedings of Machine Learning Research*, pages 31693–31715, 2023.

[38] Yuhang Wang, Yuling Shi, Shaoqiu Zhang, Jialiang Liang, et al. SWE-Pruner Pro: The coder LLM already knows what to prune. arXiv:2607.18213, 2026.

[39] Mark Weiser. Program slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.

[40] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. Describes the Agentless approach.

[41] Yue Xue. When and how context rot appears in coding agents: A white-box study of agent skills in code auditing. arXiv:2607.17937, 2026.

[42] Ori Yoran, Tomer Wolfson, Ori Ram, et al. Making retrieval-augmented language models robust to irrelevant context. arXiv:2310.01558, 2023.

[43] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, et al. Multi-SWE-bench: A multilingual benchmark for issue resolving. arXiv:2504.02605, 2025.

[44] Andreas Zeller and Ralf Hildebrandt. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, 28(2):183–200, 2002.

[45] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, pages 2471–2484, 2023.

[46] Lei Zhang, Yunshui Li, Jiaming Li, Xiaobo Xia, et al. Hierarchical context pruning: Optimizing real-world code completion with repository-level pretrained code LLMs. arXiv:2406.18294, 2024.